

CreatorCRM · Complete Implementation Plan

Version: 1.3 **Created:** 2026-04-02 **Last Updated:** 2026-06-09 **Status:** All 8 phases complete · fully functional · UI redesigned + landing page added

1. Project Overview

What

A free, self-hosted lightweight CRM + email outreach tool for the AI Creator Search campaign.

Why

- Replaces paid tools like YAMM (\$50/year) with a \$0 solution
- Sends up to **2,000 emails/day** via Gmail API (Google Workspace)
- Full **open + click tracking** via Cloudflare Workers (free tier)
- Complete control over contacts, templates, campaigns, and analytics

Who

Built for a non-technical user. Must be simple to run (`python3 app.py`) and operate via a browser dashboard.

2. Tech Stack

| Layer | Technology | Why | |-----|-----|-----| | Backend | Flask (Python) | Simple, single-file entry point, Jinja2 built-in | | Database | SQLite via SQLAlchemy | Zero setup, single file, ORM for readable queries | | Frontend | Jinja2 templates + Bootstrap 5 CDN | No build step, no npm, no React | | Email | Gmail API + google-api-python-client | Official Google library, OAuth2 built-in | | Tracking | Cloudflare Worker + D1 (SQLite at edge) | 100K req/day free, globally distributed | | Scheduling | APScheduler (in-process) | No Redis/Celery needed, runs inside Flask | | Charts | Chart.js via CDN | No build step, lightweight |

Free Tier Limits

| Service | Free Limit | Monthly Equivalent | |-----|-----|-----| | Gmail API (Workspace) | 2,000 emails/day | ~60,000/month | | Cloudflare Workers | 100,000 requests/day | ~3,000,000/month | | Cloudflare D1 | 100K writes/day, 5M reads/day | More than enough | | Total Cost | **\$0** | |

3. Project Structure

```
crm/
... PLAN.md                # This file · living implementation
plan
... PLAN.pdf               # PDF version of this plan
... app.py                 # Flask app, all routes, entry
point
... config.py              # Settings loaded from .env
... database.py            # SQLAlchemy engine, session, Base,
init_db()
... models.py              # All ORM models (7 tables)
```

```

... gmail_service.py
send_email()
... tracking.py
rewriting
... campaign_sender.py
limiting + APScheduler jobs
... csv_importer.py
import logic
... requirements.txt
... .env
... credentials.json
(gitignored)
... token.json
(gitignored)
... crm.db
(gitignored)
.
... templates/
.   ... base.html
flash messages
.   ... dashboard.html
.   ... contacts/
.       ... list.html
pagination
.   ... detail.html
history
.   ... import.html
.   ... email_templates/
.       ... list.html
.       ... editor.html
preview
.   ... campaigns/
.       ... list.html
.       ... create.html
.       ... detail.html
breakdown
.   ... review.html
sending
.   ... settings.html
config
.
... static/
.   ... style.css
Editorial Workspace)
.
... landing/
/landing/ + standalone)
.   ... index.html
markup
.   ... landing.css
design tokens)
.   ... landing.js
accordion (vanilla JS)
.   ... assets/

```

OAuth2 flow, token management,
 # Tracking pixel injection + link
 # Batch sending logic + rate
 # CSV upload, column mapping,
 # Python dependencies
 # Secrets · NEVER commit (gitignored)
 # Google OAuth credentials
 # Gmail OAuth token · auto-created
 # SQLite database · auto-created
 # Jinja2 HTML templates
 # App shell · dark sidebar layout,
 # Stats overview + recent activity
 # Contact table with search, filter,
 # Single contact view + outreach
 # CSV upload + column mapping UI
 # List all saved templates
 # Create/edit template with live
 # All campaigns with status badges
 # Campaign creation wizard
 # Campaign stats + per-contact
 # Preview personalized emails before
 # Gmail auth status, Cloudflare
 # Full design-system re-skin (Warm
 # Marketing landing page (Flask
 # The page · all sections, semantic
 # Landing styles (shares the app
 # Scroll reveals, sticky nav, FAQ
 # Real product screenshots used on

```

the page
.
... cloudflare-worker/           # Deployed to Cloudflare (separate
from Flask)
... wrangler.toml                 # Worker + D1 config
... src/
... index.js                       # Open pixel + click redirect + D1
storage

```

4. Database Schema

4.1 contact

Column	Type	Notes
id	INTEGER PK	Auto-increment
email	TEXT UNIQUE NOT NULL	Primary identifier
first_name	TEXT	
last_name	TEXT	
company	TEXT	
title	TEXT	Job title
notes	TEXT	Free-form notes
status	TEXT	active, unsubscribed, bounced
created_at	TIMESTAMP	UTC
updated_at	TIMESTAMP	UTC

4.2 tag

Column	Type	Notes
id	INTEGER PK	
name	TEXT UNIQUE NOT NULL	e.g., "brands", "tier1"

4.3 contact_tag (join table)

Column	Type	Notes
contact_id	INTEGER FK	contact
tag_id	INTEGER FK	tag
PRIMARY KEY (contact_id, tag_id)		

4.4 email_template

Column	Type	Notes
id	INTEGER PK	
name	TEXT NOT NULL	Internal name
subject	TEXT NOT NULL	Supports {first_name} etc.
body_html	TEXT NOT NULL	Full HTML, supports variables
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

4.5 campaign

Column	Type	Notes
id	INTEGER PK	
name	TEXT NOT NULL	
template_id	INTEGER FK	emailtemplate
status	TEXT	draft, sending, paused, completed
batch_size	INTEGER	Default 50
batch_delay	INTEGER	Seconds between batches, default 60
total_sent	INTEGER	Running count
total_opened	INTEGER	Running count
total_clicked	INTEGER	Running count
created_at	TIMESTAMP	
started_at	TIMESTAMP	
completed_at	TIMESTAMP	

4.6 campaign_contact (per-recipient tracking)

Column	Type	Notes
id	INTEGER PK	
campaign_id	INTEGER FK	campaign
contact_id	INTEGER FK	contact
status	TEXT	pending, sent, failed
sent_at	TIMESTAMP	
message_id	TEXT	Gmail message ID
UNIQUE (campaign_id, contact_id)		

4.7 tracking_event

Column	Type	Notes
id	INTEGER PK	
campaign_contact_id	INTEGER FK	campaign_contact
event_type	TEXT NOT NULL	open or click
url	TEXT	Original URL (for clicks only)
ip_		

address | TEXT | | user_agent | TEXT | | created_at | TIMESTAMP | |

4.8 daily_send_log

| Column | Type | Notes | |-----|-----|-----| | date_str | TEXT PK | e.g., 2026-04-02 | | count | INTEGER |
Emails sent today |

5. Key Features & How They Work

5.1 Contact Management

- **Import CSV:** Upload · see column headers · map each to a DB field via dropdowns · preview 5 rows · import
- **Duplicate handling:** Match on email. If exists, update fields (don't create duplicate)
- **Tags:** Create tags, assign to contacts. Filter contact list by tag
- **Status:** active (default), unsubscribed (from unsubscribe link), bounced (from send failure)

5.2 Email Templates

- **Editor:** HTML textarea on left, live preview iframe on right
- **Variables:** Click buttons to insert {first_name}, {last_name}, {company}, {title}, {email} at cursor position
- **Preview:** Shows template rendered with data from first contact (or dummy data)
- **Test send:** Button to send a test email to yourself

5.3 Gmail Integration

- **One-time setup:** User creates Google Cloud project, enables Gmail API, downloads credentials.json
- **OAuth flow:** Click "Connect Gmail" in Settings · browser opens Google consent screen · authorize · token saved
- **Token refresh:** Auto-refresh on each send. If refresh fails, pause campaign and show reconnect alert
- **Rate limiting:** Hard stop at 1,950 emails/day (safety margin below 2,000 limit)

5.4 Campaign Management

- **Create flow:** Name · Select template · Select contacts (by tag or manual) · Set batch size/delay · Review sample emails · Start
- **Sending:** APScheduler processes batches of batch_size emails every batch_delay seconds
- **Controls:** Pause, resume, or cancel a running campaign
- **Safety:** Review step shows 3 sample personalized emails + total recipient count with confirmation

5.5 Tracking

- **Open tracking:** 1x1 invisible pixel appended to email HTML body. Points to Cloudflare Worker
- **Click tracking:** All <a href> links rewritten to go through Cloudflare Worker, which logs and redirects
- **Token:** Each email gets a unique tracking token (base64 JSON with campaign_contact_id + HMAC)

- signature)
- **Data flow:** Worker stores events in D1 · Flask polls every 5 min · inserts into local `tracking_event` table
- **Limitations:** Apple Mail Privacy Protection inflates open rates (~50-60% of users). Click tracking is more reliable

5.6 Personalization Engine

Simple string replacement:

```
variables = {
    '{first_name}': contact.first_name or '',
    '{last_name}': contact.last_name or '',
    '{company}': contact.company or '',
    '{title}': contact.title or '',
    '{email}': contact.email,
    '{unsubscribe_url}': generate_unsubscribe_url(contact)
}
```

6. Tracking Architecture (Detailed)

Why no public server needed

Instead of the Cloudflare Worker calling back to the local Flask app (which would require exposing it to the internet), we use a **pull model**:

- 1 Email recipient opens email or clicks link · request hits Cloudflare Worker
- 2 Worker writes event to **Cloudflare D1** (SQLite at the edge)
- 3 Flask app has a background job (APScheduler, every 5 minutes) that calls `GET /api/events?since=<timestamp>&secret=<secret>` on the Worker
- 4 Worker returns new events as JSON
- 5 Flask inserts them into local `tracking_event` table
- 6 Flask calls `DELETE /api/events?before=<timestamp>` to clean up processed events

Cloudflare Worker URL scheme

`https://track.yourdomain.workers.dev/open/<tracking_token>`

`https://track.yourdomain.workers.dev/click/<tracking_token>?url=<base64_encoded_url>`

`https://track.yourdomain.workers.dev/unsubscribe/<tracking_token>`

`https://track.yourdomain.workers.dev/api/events?since=<timestamp>&secret=<secret>`

Tracking token format

`Base64({ "cc_id": 123, "hmac": "sha256_signature" })`

- `cc_id` = campaign_contact.id
 - `hmac` = HMAC-SHA256(cc_id, TRACKING_SECRET) to prevent forgery
-

7. Build Phases

Phase 1: Foundation

Files: app.py, config.py, database.py, models.py, requirements.txt, base.html, dashboard.html **Goal:** python3 app.py · see dashboard in browser at localhost:5050 **Status:** Complete

Phase 2: Contact Management

Files: csv_importer.py, contacts/list.html, contacts/detail.html, contacts/import.html **Goal:** Import CSV, view/edit/delete contacts, tag contacts, filter by tag **Status:** Complete

Phase 3: Email Templates

Files: email_templates/list.html, email_templates/editor.html **Goal:** Create/edit HTML templates with personalization variables and live preview **Status:** Complete

Phase 4: Gmail Integration

Files: gmail_service.py, settings.html **Goal:** Connect Gmail via OAuth, send test emails, see connection status **Status:** Complete

Phase 5: Campaign Management

Files: campaign_sender.py, campaigns/list.html, campaigns/create.html, campaigns/detail.html, campaigns/review.html **Goal:** Create campaign, select contacts + template, send in batches, pause/resume **Status:** Complete

Phase 6: Tracking Infrastructure

Files: tracking.py, cloudflare-worker/src/index.js, cloudflare-worker/wrangler.toml **Goal:** Open + click tracking via Cloudflare Worker, events pulled into Flask **Status:** Complete

Phase 7: Stats Dashboard

Files: Update dashboard.html, campaigns/detail.html, contacts/detail.html **Goal:** Open/click rates per campaign, per-contact history, Chart.js charts **Status:** Complete

Phase 8: Polish

Files: Various **Goal:** Unsubscribe handling, CSV export, error notifications, setup README **Status:** Complete

8. Setup Instructions (For User)

Prerequisites

- Python 3.10+ installed
- A Google Workspace account
- A Cloudflare account (free)

Step-by-step

1 Install dependencies:

```
cd crm/  
pip3 install -r requirements.txt
```

2 Google Cloud setup:

- Go to console.cloud.google.com
- Create a new project called "CreatorCRM"
- Enable the **Gmail API**
- Go to Credentials · Create OAuth 2.0 Client ID (type: Desktop Application)
- Download the JSON file, rename to `credentials.json`, place in `crm/` folder

3 Cloudflare Worker setup:

```
npm install -g wrangler  
cd cloudflare-worker/  
wrangler login  
wrangler d1 create crm-tracking  
# Copy the database_id into wrangler.toml  
wrangler deploy  
# Copy the Worker URL
```

4 Create .env file:

```
CLOUDFLARE_WORKER_URL=https://your-worker.workers.dev  
TRACKING_SECRET=any-random-string-here
```

5 Run the app:

```
python3 app.py
```

6 Open browser: Go to `http://localhost:5050`

7 Connect Gmail: Click "Connect Gmail" in Settings · authorize in browser

9. Risk Mitigation

| Risk | Mitigation | |-----|-----| | Gmail marks emails as spam | Small batches (50/min), proper MIME headers, include unsubscribe link | | OAuth token expires | Auto-refresh on each send. Pause campaign + alert if refresh fails | | SQLite concurrency | WAL mode enabled. Flask dev server is single-threaded | | Cloudflare free tier limits | 100K req/day is 50x our volume. More than enough | | Accidental mass send | Review step shows sample emails + recipient count + confirmation dialog | | Tracking pixel blocked | Apple Mail inflates opens. Click tracking is more reliable signal |

10. UI Design System · "Warm Editorial Workspace"

A cohesive design language applied across the whole app and the landing page. **Reuse these tokens for any new UI** so everything stays consistent.

Typography

| Role | Font | |-----|-----| | Display / headings / large numbers | **Fraunces** (variable serif, optical sizing) | | UI / body text | **Hanken Grotesk** | | Numerals / counts / mono | **Spline Sans Mono** |

Palette

| Token | Hex | Use | |-----|-----|-----| | Paper | #F6F2EA | App/page background (warm, not white) | | Ink | #1C1A16 | Text + the dark sidebar/comparison sections | | Ember | #E8482B | Primary accent · buttons, active nav, links | | Forest | #2E6F50 | Success / positive metrics | | Gold | #BE8A1E | Warnings | | Slate | #2F5C84 | Info / "opens" |

Layout

- **App chrome:** a dark ink **left sidebar** (grouped nav: Workspace / Outreach / System)
 - a warm paper content area with a frosted sticky topbar. Defined in `templates/base.html` + `static/style.css`. Fully responsive (sidebar collapses to a slide-in drawer on mobile).
 - **Components:** the app keeps Bootstrap 5 markup but `static/style.css` re-skins cards, buttons, tables, badges, forms, progress bars, alerts, modals and dropdowns to the tokens above · so existing templates restyle automatically.
-

11. Marketing Landing Page

A public marketing page (`landing/`) positioning CreatorCRM as the **free, self-hosted challenger** to paid mail-merge tools.

- **Delivery:** served by Flask at `/landing/` (routes in `app.py` via `send_from_directory`), and also fully **standalone** · `landing/` is self-contained static files hostable on Cloudflare Pages / Netlify / GitHub Pages.
- **Stack:** plain `index.html` + `landing.css` + `landing.js` (vanilla JS: IntersectionObserver scroll-reveals, sticky-nav shrink, mobile menu, FAQ accordion). No build step. Reuses the §10 design tokens.
- **Sections:** sticky nav · hero (with product screenshot) · tech trust strip · problem/features grid · how-it-works · product showcase **comparison table (centerpiece)** · pricing punchline · FAQ · CTA band · footer.
- **Comparison data (as of June 2026):** YAMM \$25·50/yr · Mailmeteor \$13·25/mo · Gmail \$30·60/mo · CreatorCRM \$0. Footnoted that the real send cap is Gmail's own (·500/day personal, 2,000/day Workspace), not the tool.
- **Primary CTA:** "Launch CreatorCRM" · the live app(`/`).

Status / TODOs

- · Published to GitHub <https://github.com/gsp9145/creator-crm> · landing page "View source" / GitHub links now point there.
 - · Swap `landing/assets/` screenshots for **demo data** if desired (they currently show real account/contact data · the owner confirmed this is acceptable for now).
-

12. Changelog

| Date | Version | Changes | |-----|-----|-----| | 2026-04-02 | 1.0 | Initial plan created | | 2026-04-04 | 1.1 | All 8 phases built and tested. All 12 routes passing. | | 2026-04-04 | 1.2 | Security + deliverability hardening: CSRF protection (Flask-WTF), email validation, auto-generated secrets, file-based CSV import, atomic daily send counter, retry logic for failed sends, CAN-SPAM headers (List-Unsubscribe, physical address), domain warm-up support, deduplicated tracking counters, fixed nested form bug, removed open redirect. | | 2026-06-09 | 1.3 | Full UI re-skin "Warm Editorial Workspace" design system (Fraunces + Hanken Grotesk, ember accent) with a dark sidebar layout

(§10). Added a marketing landing page at `/landing/`, also standalone-hostable (§11). Fixed stale `localhost:5000` · `5050` references. |